# Strings Notes

- In Python, a string is a sequence of characters written inside quotes.
- It can include letters, numbers, symbols, and spaces.

## Creating a String

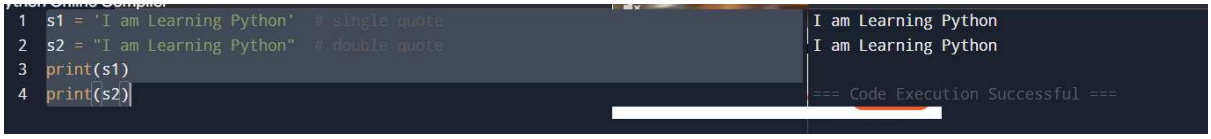
Strings can be created using either **single ('...')** or **double ("...")** quotes.

s1 = 'I am Learning Python'  # single quote

s2 = "I am Learning Python"  # double quote

print(s1)

print(s2)




## Multi-line Strings

Use triple quotes **('''...''' )** or **( """...""")** for strings that span multiple lines.

s = """I am Learning

Python String on GeeksforGeeks"""

print(s)

```
s = '''I'm a
Geek'''
print(s)
```

# How to Access Characters of a String?

We can access characters of a string by using the following ways.

1) By using index

2) By using slice operator

## 1. Accessing Characters By using Index:

Python supports both +ve and -ve Index.

- +ve Index means Left to Right (Forward Direction)
- -ve Index means Right to Left (Backward Direction)

S = “Python”

| **Positive Index** | **0** | **1** | **2** | **3** | **4** | **5** |
|---|---|---|---|---|---|---|
| **Character** | **P** | **Y** | **T** | **H** | **O** | **N** |
| **Negative Index** | -6 | -5 | -4 | -3 | -2 | -1 |

Print(s[3]) → H

Print(s[-5]) → Y

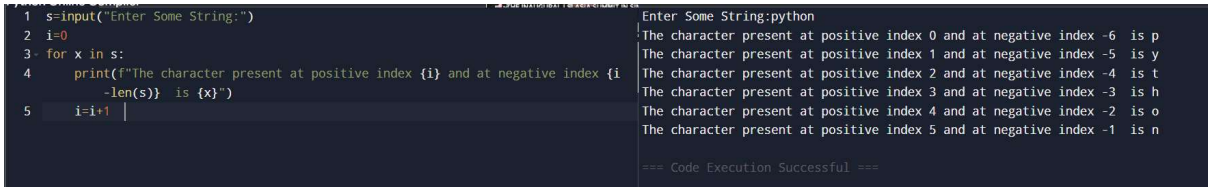
Print(s[50]) → IndexError: string index out of range

## QUES:

**Write a Program to Accept some String from the Keyboard and display its Characters by Index wise (both Positive and Negative Index)?**

**User given input value as S = “Python”**

```
s=input("Enter Some String:")
i=0
for x in s:
    print(f"The character present at positive index {i} and at negative index {i-len(s)}  is {x}")
    i=i+1
```




## 2. Accessing Characters by using Slice Operator:

Syntax: s[begin_index:end_index:step]

- **Begin Index: From where we have to consider slice (substring)**
- **End Index: We have to terminate the slice (substring) at endindex-1**
- **Step: Incremented Value.**

**Note:**

- **If we are not specifying begin index then it will consider from beginning of the string.**
- **If we are not specifying end index then it will consider up to end of the string.**
- **The default value for step is 1.**

Using the string `s = "PYTHON"`:

| Operation | Result | Explanation |
|---|---|---|
| `s[1:4]` | `"YTH"` | Starts at index 1 ('Y') and stops before index 4 ('O'). |
| `s[:3]` | `"PYT"` | Omitted `start` defaults to 0; gets first 3 characters. |
| `s[2:]` | `"THON"` | Omitted `stop` goes until the end of the string. |
| `s[::2]` | `"PTO"` | Uses a `step` of 2 to skip every other character. |
| `s[::-1]` | `"NOHTYP"` | A negative `step` reverses the entire string. |
| `word[::3]` | `"PH"` | Index 0, then skip 3 to Index 3 |
| `word[-4:-1]` | `"THO"` | From 4th-to-last up to (but not including) the last |

## Behaviour of Slice Operator:

**1) s[begin:end:step]**

**2) Step value can be either +ve or –ve**

**3) If +ve then it should be forward direction(left to right) and we have to consider begin to end-1**

**4) If -ve then it should be backward direction (right to left) and we have to consider begin to end+1.**

Note:

- In the backward direction if end value is -1 then result is always empty.
- In the forward direction if end value is 0 then result is always empty.

```
s = 'abcdefghij'
print(s[1:6:2])
print(s[::1])
print(s[::-1])
print(s[3:7:-1])
print(s[7:4:-1])
print(s[0:10000:1])
print(s[-4:1:-1])
print(s[-4:1:-2])
print(s[5:0:1])
print(s[9:0:0])
print(s[0:-10:-1])
print(s[0:-11:-1])
print(s[0:0:1])
print(s[0:-9:-2])
print(s[-5:-9:-2])
print(s[10:-1:-1])
print(s[10000:2:-1])
```

| Code | Output | Explanation |
|---|---|---|
| `s[1:6:2]` | `'bdf'` | Indices 1, 3, 5. |
| `s[::1]` | `'abcdefghij'` | Standard copy of the string. |
| `s[::-1]` | `'jihgfedcba'` | Reverses the string. |
| `s[3:7:-1]` | `''` | **Empty:** You can't go from 3 to 7 using a negative step. |
| `s[7:4:-1]` | `'hgf'` | Indices 7, 6, 5 (Stops *before* 4). |
| `s[0:10000:1]` | `'abcdefghij'` | Python is "forgiving"—out-of-bounds indices just stop at the end. |
| `s[-4:1:-1]` | `'gfedc'` | From 'g' (index 6) back to index 2. |
| `s[-4:1:-2]` | `'gec'` | Every second char from 'g' back to index 2. |
| `s[5:0:1]` | `''` | **Empty:** You can't go from 5 to 0 with a positive step. |
| `s[9:0:0]` | **ValueError** | **Error:** Step cannot be zero! |
| `s[0:-10:-1]` | `''` | **Empty:** Index 0 is 'a', -10 is also 'a'. No room to move. |
| `s[0:-11:-1]` | `'a'` | Starts at 0, goes back to the very beginning. |
| `s[0:0:1]` | `''` | **Empty:** Start and stop are the same. |
| `s[0:-9:-2]` | `''` | **Empty:** Cannot go backwards from 0 to -9 with a negative step. |
| `s[-5:-9:-2]` | `'fd'` | Starts at 'f' (-5), jumps to 'd' (-7). |
| `s[10:-1:-1]` | `''` | **Empty:** Start index 10 is at the end; cannot go to -1 ('j') backwards. |
| `s[10000:2:-1]` | `'jihgfed'` | Starts at the end, goes back to index 3 ('d'). |

## len() in-built Function:

**We can use len() function to find the number of characters present in the string.**

**S = “python”**

**Print(len(s)) → 6**

## String Concatenation & String repetition

+ -> used for concatenation

*  -> repetition

print("python"+"java") → pythonjava

print("java"*2) → javajava

## Comparison of Strings:

```
s1=input("Enter first string:")
s2=input("Enter Second string:")
if s1==s2:
   print("Both strings are equal")
elif s1<s2:
   print("First String is less than Second String")
else:
   print("First String is greater than Second String")
```

### Notes:

When you run this code with those specific inputs, Python compares the strings based on the **Unicode (ASCII) values** of their characters, starting from the very first letter.

First string = ‘pppp’

Second string = ‘dddddd’

- **s1:** `"pppp"` (Starts with `'p'`, which has a decimal value of **112**)
- **s2:** `"ddddddd"` (Starts with `'d'`, which has a decimal value of **100**)

Because `112` is greater than `100`, Python immediately determines that **"p" > "d"**.

## Removing Spaces from the String:

We can use the following 3 methods

1) rstrip() → To remove spaces at right hand side

2) lstrip() → To remove spaces at left hand side

3) strip() → To remove spaces both sides

```
# A string with messy spaces on both ends
text = "    Python Programming    "

# 1) rstrip() -> Removes spaces from the RIGHT only
print(f"'{text.rstrip()}'")
# Output: '    Python Programming'

# 2) lstrip() -> Removes spaces from the LEFT only
print(f"'{text.lstrip()}'")
# Output: 'Python Programming    '

# 3) strip() -> Removes spaces from BOTH ends
print(f"'{text.strip()}'")
# Output: 'Python Programming'
```

## Finding Substrings:

**We can use the following 4 methods.**

**For forward direction:**

**1) find()**

**2) index()**

**For backward direction:**

**1) rfind()**

**2) rindex()**

# find():

s.find(substring)

- Returns index of first occurrence of the given substring.
- If it is not available then we will get -1.

**Example:**

```
s = "learning Python is fun"
print(s.find("Python"))  # Output: 9 (Starts at index 9)
print(s.find("Java"))    # Output: -1 (Not found)
```

*Note:*
*By default find() method can search total string. We can also specify the boundaries to search.*
*s.find(substring,begin,end)*
*It will always search from begin index to end-1 index.*

```
s = "ABRACADABRA"

# 1. Searching the whole string
print(s.find("ABR"))            # Output: 0 (The very first ABR)

# 2. Searching starting from index 1
print(s.find("ABR", 1))         # Output: 7 (Skips the first one, finds the second)

# 3. Searching within a tight window [1:8]
# It looks at: "BRACADA"
print(s.find("ABR", 1, 8))      # Output: -1 (Found nothing! 'ABR' starts at 7 but
ends at 10)

# 4. Searching within index [1:10]
# It looks at: "BRACADABR"
print(s.find("ABR", 1, 10))     # Output: 7 (Found it because the full word fits)
```

**index():**

index() method is exactly same as find() method except that if the specified substring is not available then we will get ValueError.

```
s = "Python Programming"

# Case 1: Substring exists
print(s.index("Python"))  # Output: 0

# Case 2: Substring is missing
# print(s.index("Java"))  # CRASHES with ValueError: substring not found
```

## Counting substring in the given String:

```
s = "abcabcabc"
print(s.count("abc"))  # Output: 3
print(s.count("a"))    # Output: 3
print(s.count("z"))    # Output: 0
```

## Replacing a String with another String:

s.replace(oldstring, newstring)

```
s = "Learning Python is very difficult"
s1 = s.replace("difficult","easy")
print(s1)
```

---

```
s = "ababababababab"
s1 = s.replace("a","b")
print(s1)
Output: bbbbbbbbbbbbbb
```

---

**Ques:**

**String Objects are Immutable then how we can change the Content by using replace() Method?**

Once we creates string object, we cannot change the content.
This non changeable behaviour is nothing but immutability. If we are trying to change the content by using any method, then with those changes a new object will be created and changes won't be happened in existing object.

- Hence with replace() method also a new object got created but existing object won't be changed.

**Eg:**

```
s = "abab"
s1 = s.replace("a","b")
print(s,"is available at :",id(s))
print(s1,"is available at :",id(s1))
```

**Output:**

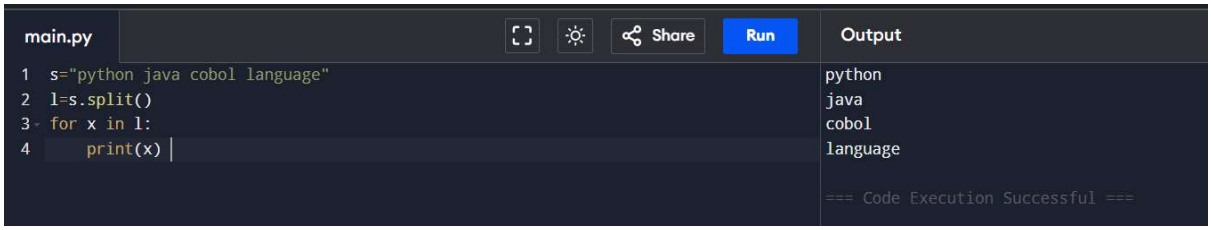
abab is available at : 4568672
bbbb is available at : 4568704

In the above example, original object is available and we can see new object which was created because of replace() method.

# Splitting of Strings:

- We can split the given string according to specified seperator by using split() method.
- l = s.split(seperator)
- The default seperator is space. The return type of split() method is List.

```
s="python java cobol language"
l=s.split()
for x in l:
    print(x)
```




```
s="12-02-2026"
l=s.split('-')
for x in l:
    print(x)
```

## Joining of Strings:

We can join a Group of Strings (List OR Tuple) wrt the given Seperator.

s = seperator.join(group of strings)

Example:

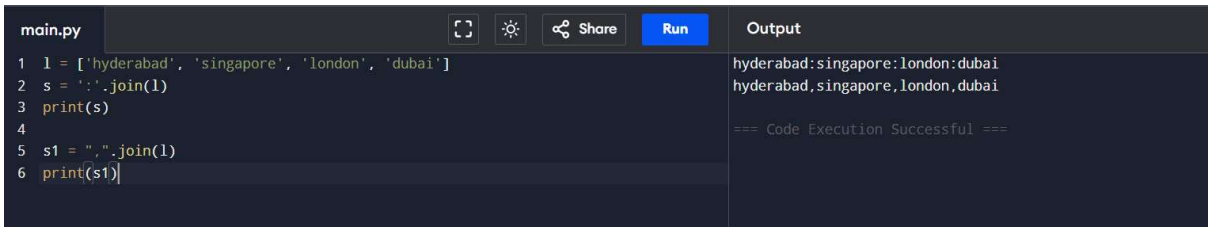
l = ['hyderabad', 'singapore', 'london', 'dubai']

s = ':'.join(l)

print(s)

s1 = ",".join(l)

print(s1)



## Changing Case of a String:

We can change case of a string by using the below methods.

a. upper() →To convert all characters to upper case
b. lower() → To convert all characters to lower case
c. swapcase() → Converts all lower case characters to upper case and all upper case characters to lower case
d. title() →To convert all character to title case. i.e first character in every word should be upper case and all remaining characters should be in lower case.
e. capitalize() →Only first character will be converted to upper case and all remaining characters can be converted to lower case

## Example:

```
text = "heLLo pYthon WORld"

# a. upper() - All characters to upper case

print(f"upper():     {text.upper()}")

# b. lower() - All characters to lower case

print(f"lower():     {text.lower()}")

# c. swapcase() - Swaps case of each character

print(f"swapcase():   {text.swapcase()}")

# d. title() - First character of every word to upper case

print(f"title():     {text.title()}")

# e. capitalize() - Only the very first character to upper case

print(f"capitalize(): {text.capitalize()}")
```

```python
text = "heLLo pYthon WORld"

# a. upper() - All characters to upper case
print(f"upper():      {text.upper()}")

# b. lower() - All characters to lower case
print(f"lower():      {text.lower()}")

# c. swapcase() - Swaps case of each character
print(f"swapcase():   {text.swapcase()}")

# d. title() - First character of every word to upper case
print(f"title():      {text.title()}")

# e. capitalize() - Only the very first character to upper case
print(f"capitalize(): {text.capitalize()}")
```

```
upper():      HELLO PYTHON WORLD
lower():      hello python world
swapcase():   HEllO PyTHON worLD
title():      Hello Python World
capitalize(): Hello python world

=== Code Execution Successful ===
```

## Checking Starting and Ending Part of the String:

**Python contains the following methods for this purpose.**

**1) s.startswith(substring)**

**2) s.endswith(substring)**

## Example:

```python
s = 'learning Python is very easy'
print(s.startswith('learning'))
print(s.endswith('learning'))
print(s.endswith('easy'))
```

```
True
False
True

=== Code Execution Successful
```

# To Check Type of Characters Present in a String:

Python contains the following methods for this purpose.

1) isalnum(): Returns True if all characters are alphanumeric( a to z , A to Z ,0 to9 )

2) isalpha(): Returns True if all characters are only alphabet symbols(a to z,A to Z)

3) isdigit(): Returns True if all characters are digits only( 0 to 9)

4) islower(): Returns True if all characters are lower case alphabet symbols

5) isupper(): Returns True if all characters are upper case alphabet symbols

6) istitle(): Returns True if string is in title case

7) isspace(): Returns True if string contains only spaces

**Example:**

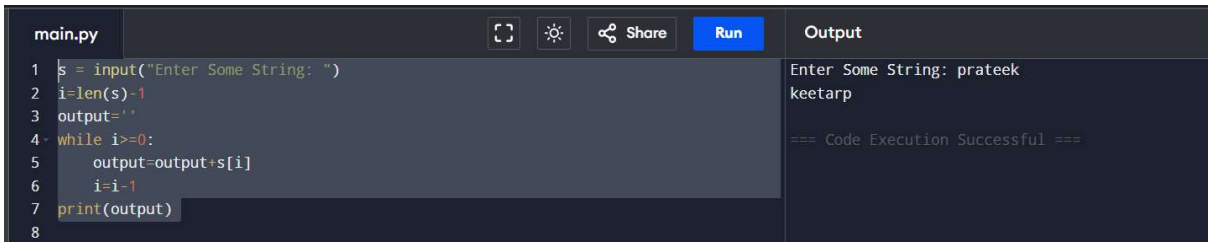
```
s1 = "Python3"

s2 = "Python"

s3 = "12345"

s4 = "hello"

s5 = "HELLO"

s6 = "Python Is Cool"

s7 = "   "

# 1) print(s1.isalnum())  # True

# 2) print(s2.isalpha())  # True

# 3) print(s3.isdigit())  # True

# 4) print(s4.islower())  # True

# 5) print(s5.isupper())  # True

# 6) print(s6.istitle())  # True

# 7) print(s7.isspace())  # True
```

**Ques:**

**Write a Program to Reverse the given String?**

## 1st Way:

```
s = input("Enter Some String:")
print(s[::-1])
```

## 2nd Way:

```
s = input("Enter Some String: ")
i=len(s)-1
output=''
while i>=0:
    output=output+s[i]
    i=i-1
print(output)
```